Introduction to the Kotlin Language

June 17, 2026 2:43 PM

https://www.baeldung.com/kotlin/intro

Introduction to the Kotlin Language

Last updated: March 19, 2024

Written by:
baeldung

Reviewed by:
Grzegorz Piwowarek

Kotlin Basics

1. Overview

In this tutorial, we're going to take a look at Kotlin, a new language in the JVM world, and some of its basic features, including classes, inheritance, conditional statements, and looping constructs.

Then we will look at some of the main features that make Kotlin an attractive language, including null safety, data classes, extension functions, and *String* templates.

2. Maven Dependencies

To use Kotlin in your Maven project, you need to add the Kotlin standard library to your *pom.xml*:

```
<dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-stdlib</artifactId>
    <version>1.0.4</version>
</dependency>
```

To add JUnit support for Kotlin, you will also need to include the following dependencies:

```
<dependency>
    <groupId>org.jetbrains.kotlin</groupId>
    <artifactId>kotlin-test-junit</artifactId>
    <version>1.0.4</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>
</dependency>
```

You can find the latest versions of kotlin-stdlib, kotlin-test-junit, and junit on Maven Central.

Finally, you will need to configure the source directories and Kotlin plugin in order to perform a Maven build:

```
<build>
    <sourceDirectory>${project.basedir}/src/main/kotlin</sourceDirectory>
    <testSourceDirectory>${project.basedir}/src/test/kotlin</testSourceDirectory>
    <plugins>
        <plugin>
```

```
<build>
    <sourceDirectory>${project.basedir}/src/main/kotlin</sourceDirectory>
    <testSourceDirectory>${project.basedir}/src/test/kotlin</testSourceDirectory>
    <plugins>
        <plugin>
            <artifactId>kotlin-maven-plugin</artifactId>
            <groupId>org.jetbrains.kotlin</groupId>
            <version>1.0.4</version>
            <executions>
                <execution>
                    <id>compile</id>
                    <goals>
                        <goal>compile</goal>
                    </goals>
                </execution>
                <execution>
                    <id>test-compile</id>
                    <goals>
                        <goal>test-compile</goal>
                    </goals>
                </execution>
            </executions>
        </plugin>
    </plugins>
</build>
```

You can find the latest version of kotlin-maven-plugin in Maven Central.

3. Basic Syntax

Let's look at the basic building blocks of Kotlin Language.

There is some similarity to Java (e.g. defining packages is in the same way). Let's take a look at the differences.

3.1. Defining Functions

Let's define a Function having two Int parameters with *Int* return type:

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
```

3.2. Defining Local Variables

Assign-once (read-only) local variable:

```
val a: Int = 1
val b = 1
val c: Int
c = 1
```

Note that type of a variable *b* is inferred by a Kotlin compiler. We could also define mutable variables:

```
var x = 5
x += 1
```

4. Optional Fields

Kotlin has a basic syntax for defining a field that could be nullable (optional). When we want to declare that type of field is nullable we need to use type suffixed with a question mark:

Kotlin has a basic syntax for defining a field that could be nullable (optional). When we want to declare that type of field is nullable we need to use type suffixed with a question mark:

```
val email: String?
```

When you defined nullable field it is perfectly valid to assign a *null* to it:

```
val email: String? = null
```

That means that in an email field could be a *null*. If we will write:

```
val email: String = "value"
```

Then we need to assign a value to email field in the same statement that we declare email. It can not have a null value. We will get back to Kotlin *null* safety in a later section.

5. Classes

Let's demonstrate how to create a simple class for managing a specific category of a product. Our *ItemManager* class below has a default constructor that populates two fields — *categoryId* and *dbConnection* — and an optional *email* field:

```
class ItemManager(val categoryId: String, val dbConnection: String) {
    var email = ""
    // ...
}
```

That *ItemManager(…)* construct creates constructor and two fields in our class: *categoryId* and *dbConnection*

Note that our constructor uses the *val* keyword for its arguments — this means that the corresponding fields will be *final* and immutable. If we had used the *var* keyword (as we did when defining the *email* field), then those fields would be mutable.

Let's create an instance of ItemManager using the default constructor:

```
ItemManager("cat_id", "db://connection")
```

We could construct *ItemManager* using named parameters. It is very useful when you have like in this example function that takes two parameters with the same type e.g. *String*, and you do not want to confuse an order of them. Using naming parameters you can explicitly write which parameter is assigned. In class *ItemManager* there are two fields, *categoryId* and *dbConnection* so both can be referenced using named parameters:

```
ItemManager(categoryId = "catId", dbConnection = "db://Connection")
```

It is very useful when we need to pass more arguments to a function.

If you need additional constructors, you would define them using the *constructor* keyword. Let's define another constructor that also sets the *email* field:

```
constructor(categoryId: String, dbConnection: String, email: String)
  : this(categoryId, dbConnection) {
    this.email = email
}
```

Note that this constructor invokes the default constructor that we defined above before setting the email field. And since we already defined *categoryId* and *dbConnection* to be immutable using the *val* keyword in the default constructor, we do not need to repeat the *val* keyword in the additional constructor.

Now, let's create an instance using the additional constructor:

```
ItemManager("cat_id", "db://connection", "foo@bar.com")
```

```
ItemManager("cat_id", "db://connection", "foo@bar.com")
```

If you want to define an instance method on *ItemManager*, you would do so using the *fun* keyword:

```
fun isFromSpecificCategory(catId: String): Boolean {
    return categoryId == catId
}
```

6. Inheritance

By default, Kotlin's classes are closed for extension — the equivalent of a class marked *final* in Java.

In order to specify that a class is open for extension, you would use the *open* keyword when defining the class.

Let's define an *Item* class that is open for extension:

```
open class Item(val id: String, val name: String = "unknown_name") {
    open fun getIdOfItem(): String {
        return id
    }
}
```

Note that we also denoted the *getIdOfItem()* method as open. This allows it to be overridden.

Now, let's extend the *Item* class and override the *getIdOfItem()* method:

```
class ItemWithCategory(id: String, name: String, val categoryId: String) : Item(id, name) {
    override fun getIdOfItem(): String {
        return id + name
    }
}
```

7. Conditional Statements

In Kotlin, conditional statement *if* is an equivalent of a function that returns some value. Let's look at an example:

```
fun makeAnalyisOfCategory(catId: String): Unit {
    val result = if (catId == "100") "Yes" else "No"
    println(result)
}
```

In this example, we see that if *catId* is equal to "100" conditional block returns "Yes" else it returns "No". Returned value gets assigned to *result*.

You could create a normal *if-else* block:

```
val number = 2
if (number < 10) {
    println("number less that 10")
} else if (number > 10) {
    println("number is greater that 10")
}
```

Kotlin has also a very useful *when* command that acts as an advanced switch statement:

```
val name = "John"
when (name) {
    "John" -> println("Hi man")
    "Alice" -> println("Hi lady")
}
```

```
    "Alice" -> println("Hi lady")
}
```

8. Collections

There are two types of collections in Kotlin: mutable and immutable. When we create immutable collection it means that is read-only:

```
val items = listOf(1, 2, 3, 4)
```

There is no add function element on that list.

When we want to create a mutable list that could be altered, we need to use *mutableListOf()* method:

```
val rwList = mutableListOf(1, 2, 3)
rwList.add(5)
```

A mutable list has *add()* method so we could append an element to it. There are also equivalent method to other types of collections: *mutableMapOf(), mapOf(), setOf(), mutableSetOf()*

9. Exceptions

The mechanism of exception handling is very similar to the one in Java.

All exception classes extend *Throwable*. The exception must have a message, stacktrace, and an optional cause. **Every exception in Kotlin is unchecked**, meaning that compiler does not force us to catch them.

To throw an exception object, we need to use the throw-expression:

```
throw Exception("msg")
```

Handling of exception is done by using *try..catch block(finally* optional):

```
try {

}
catch (e: SomeException) {

}
finally {

}
```

10. Lambdas

In Kotlin, we could define lambda functions and pass them as arguments to other functions.

Let's see how to define a simple lambda:

```
val sumLambda = { a: Int, b: Int -> a + b }
```

We defined *sumLambda* function that takes two arguments of type *Int* as an argument and returns *Int*.

We could pass a lambda around:

```
@Test
fun givenListOfNumber_whenDoingOperationsUsingLambda_shouldReturnProperResult() {
    // given
    val listOfNumbers = listOf(1, 2, 3)

    // when
    val sum = listOfNumbers.reduce { a, b -> a + b }
```

```
    val listOfNumbers = listOf(1, 2, 3)

    // when
    val sum = listOfNumbers.reduce { a, b -> a + b }

    // then
    assertEquals(6, sum)
}
```

11. Looping Constructs

In Kotlin, looping through collections could be done by using a standard *for..in* construct:

```
val numbers = arrayOf("first", "second", "third", "fourth")
```

```
for (n in numbers) {
    println(n)
}
```

If we want to iterate over a range of integers we could use a range construct:

```
for (i in 2..9 step 2) {
    println(i)
}
```

Note that the range in the example above is inclusive on both sides. The *step* parameter is optional and is equivalent to incrementing the counter twice in each iteration. The output will be following:

```
2
4
6
8
```

We could use a *rangeTo()* function that is defined on *Int* class in the following way:

```
1.rangeTo(10).map{ it * 2 }
```

The result will contain (note that *rangeTo()* is also inclusive):

```
[2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

12. Null Safety

Let's look at one of the key features of Kotlin – null safety, that is built into the language. To illustrate why this is useful, we will create a simple service that returns an *Item* object:

```
class ItemService {
    fun findItemNameForId(id: String): Item? {
        val itemId = UUID.randomUUID().toString()
        return Item(itemId, "name-$itemId");
    }
}
```

The important thing to notice is returned type of that method. It is an object followed by a question mark. It is a construct from Kotlin language, meaning that *Item* returned from that method could be null. We need to handle that case at compile-time, deciding what we want to do with that object (it is more or less equivalent to Java 8 *Optional<T>* type).

If the method signature has type without question mark:

```
fun findItemNameForId(id: String): Item
```

If the method signature has type without question mark:

```
fun findItemNameForId(id: String): Item
```

then calling code will not need to handle a null case because it is guaranteed by the compiler and Kotlin language, that returned object can not be null.

Otherwise, **if there is a nullable object passed to a method, and that case is not handled, it will not compile.**

Let's write a test case for Kotlin type-safety:

```
val id = "item_id"
val itemService = ItemService()

val result = itemService.findItemNameForId(id)

assertNotNull(result?.let { it -> it.id })
assertNotNull(result!!.id)
```

We are seeing here that after executing method *findItemNameForId()*, the returned type is of Kotlin *Nullable*. To access a field of that object (*id*), we need to handle that case at compile time. Method *let()* will execute only if a result is non-nullable. *Id* field can be accessed inside of a lambda function because it is null safe.

Another way to access that nullable object field is to use Kotlin operator *!!*. It is equivalent to:

```
if (result == null){
    throwNpe();
}
return result;
```

Kotlin will check if that object is a *null* if so, it will throw a *NullPointerException*, otherwise, it will return a proper object. Function *throwNpe()* is a Kotlin internal function.

13. Data Classes

A very nice language construct that could be found in Kotlin is data classes (it is equivalent to "case class" from Scala language). The purpose of such classes is to only hold data. In our example we had an *Item* class that only holds the data:

```
data class Item(val id: String, val name: String)
```

The compiler will create for us methods *hashCode()*, *equals()*, and *toString()*. It is good practice to make data classes immutable, by using a *val* keyword. Data classes could have default field values:

```
data class Item(val id: String, val name: String = "unknown_name")
```

We see that *name* field has a default value "unknown_name".

14. Extension Functions

Suppose that we have a class that is a part of 3rd party library, but we want to extend it with an additional method. **Kotlin allows us to do this by using extension functions.**

Let's consider an example in which we have a list of elements and we want to take a random element from that list. We want to add a new function *random()* to 3rd party *List* class.

Here's how it looks like in Kotlin:

```
fun <T> List<T>.random(): T? {
    if (this.isEmpty()) return null
    return get(ThreadLocalRandom.current().nextInt(count()))
}
```

```
    if (this.isEmpty()) return null
    return get(ThreadLocalRandom.current().nextInt(count()))
}
```

The most important thing to notice here is a signature of the method. The method is prefixed with a name of the class that we are adding this extra method to.

Inside the extension method, we operate on a scope of a list, therefore using *this* gave use access to list instance methods like *isEmpty()* or *count()*. Then we are able to call *random()* method on any list that is in that scope:

```
fun <T> getRandomElementOfList(list: List<T>): T? {
    return list.random()
}
```

We created a method that takes a list and then executes custom extension function *random()* that was previously defined. Let's write a test case for our new function:

```
val elements = listOf("a", "b", "c")

val result = ListExtension().getRandomElementOfList(elements)

assertTrue(elements.contains(result))
```

The possibility of defining functions that "extends" 3rd party classes is a very powerful feature and can make our code more concise and readable.

15. String Templates

A very nice feature of Kotlin language is a possibility to use templates for *String*s. It is very useful because we do not need to concatenate *String*s manually:

```
val firstName = "Tom"
val secondName = "Mary"
val concatOfNames = "$firstName + $secondName"
val sum = "four: ${2 + 2}"
```

We can also evaluate an expression inside the *${}* block:

```
val itemManager = ItemManager("cat_id", "db://connection")
val result = "function result: ${itemManager.isFromSpecificCategory("1")}"
```

16. Kotlin/Java Interoperability

Kotlin – Java interoperability is seamlessly easy. Let's suppose that we have a Java class with a method that operates on *String*:

```
class StringUtils{
    public static String toUpperCase(String name) {
        return name.toUpperCase();
    }
}
```

Now we want to execute that code from our Kotlin class. We only need to import that class and we could execute the java method from Kotlin without any problems:

```
val name = "tom"

val res = StringUtils.toUpperCase(name)

assertEquals(res, "TOM")
```

```
val res = StringUtils.toUpperCase(name)

assertEquals(res, "TOM")
```

As we see, we used java method from Kotlin code.

Calling Kotlin code from a Java is also very easy. Let's define a simple Kotlin function:

```
class MathematicsOperations {
    fun addTwoNumbers(a: Int, b: Int): Int {
        return a + b
    }
}
```

Executing *addTwoNumbers()* from Java code is very easy:

```
int res = new MathematicsOperations().addTwoNumbers(2, 4);

assertEquals(6, res);
```

We see that call to Kotlin code was transparent to us.

When we define a method in java that return type is a *void*, in Kotlin returned value will be of a *Unit* type.

There are some special identifiers in Java language (*is*, *object*, *in*, ..) that when used in Kotlin code needs to be escaped. For example, we could define a method that has a name *object()* but we need to remember to escape that name as this is a special identifier in java:

```
fun `object`(): String {
    return "this is object"
}
```

Then we could execute that method:

```
`object`()
```

17. Conclusion

This article makes an introduction to Kotlin language and it's key features. It starts by introducing simple concepts like loops, conditional statements, and defining classes. Then shows some more advanced features like extension functions and null safety.

The code backing this article is available on GitHub. Once you're **logged in as a Baeldung Pro Member**, start learning and coding on the project.